SignBridge

AI-Powered Real-Time Sign Language Recognition & Offline Translation Assistant

Complete Comprehensive Project Guide, Architectural Specification & Viva Master Manual
Prepared for College Project Presentations, Academic Defense & Technical Viva Voce

---

Academic Subject: AI / Computer Vision Project	**Target Frameworks:** OpenCV, MediaPipe, Scikit-Learn, Streamlit
Language: Python 3.11	**Vocabulary:** 31 Classes (26 Alphabet + 5 Word Gestures)
Translation: Offline MarianMT (English to Hindi)	**TTS Engine:** Dual Offline (pyttsx3 + HTML5 Web Speech)

1. Executive Summary & Problem Statement

Over 63 million people in India suffer from significant auditory impairment, relying predominantly on Indian Sign Language (ISL) for daily communication. Unfortunately, fewer than 1% of the general hearing population understands sign language, creating severe communication barriers in healthcare, banking, education, and social environments.

Existing Solutions & Their Flaws: Most commercial translation tools rely on paid cloud APIs (e.g. Google Cloud Vision, OpenAI Whisper/Vision). These suffer from high recurring costs, critical privacy vulnerabilities (uploading private video streams to 3rd-party servers), and complete failure in low-connectivity or rural environments.

The SignBridge Solution: SignBridge is a 100% offline, privacy-first assistive platform that runs directly on commodity laptops without external GPUs or cloud subscriptions. It captures standard 30 FPS webcam video, extracts 21 skeletal hand landmarks via MediaPipe Tasks, applies translation- and scale-invariant geometric normalization, classifies the gesture using a Random Forest classifier (99.0% accuracy), debounces tokens into fluent sentences, vocalizes text via dual Text-to-Speech engines, and translates English sentences into Devanagari Hindi via a local MarianMT neural network.

> **■ Mentor Note: The Core Philosophy**
>
> **In Simple Words (Hinglish):** SignBridge ek aisa web assistant hai jo deaf ya mute logon ke hath ke signs ko laptop camera se dekh kar real-time me pehchanta hai. Ye na sirf signs ko text me jodta hai, balki use bol kar sunata hai (voice audio) aur saath hi bina internet ke Hindi me translate bhi kar deta hai!

2. Complete Technology Stack & Architecture

Every component in SignBridge was chosen with precise engineering purpose to guarantee zero subscription costs and sub-50ms inference latency.

Technology	Purpose in SignBridge	Working Mechanism	Cost
Python 3.11	Core programming runtime	Compiles backend logic & bridges ML libraries	Free / OSS
OpenCV	Webcam video acquisition & overlay	Reads 30 FPS frames & draws hand landmark skeleton	Free / OSS
MediaPipe Tasks	21 3D hand landmark detection	Loads hand_landmarker.task; outputs x,y,z coordinates	Free (Google)

NumPy	Geometric math & normalization	Computes Euclidean distances & 63D feature vectors	Free / OSS
Pandas	Dataset storage & management	Reads/writes data/isl_dataset.csv with session metadata	Free / OSS
Scikit-Learn	Random Forest classifier	Ensemble of 150 Decision Trees classifying 31 gestures	Free / OSS
Joblib	Model persistence & loading	Serializes trained model artifact to models/isl_classifier.joblib	Free / OSS
Streamlit	Frontend UI & routing	Reactive web interface with dark/light themes & session state	Free / OSS
MarianMT (Transformers)	Offline English-to-Hindi translation	Local Seq2Seq neural network (Helsinki-NLP/opus-mt-en-hi)	Free / OSS
pyttsx3 & Web Speech	Dual Text-to-Speech audio	Threaded native OS voice + browser HTML5 synthesis	Free / Local
Pytest	Automated test suite (52 tests)	Validates invariance, bounds, dataset schema & routing	Free / OSS

3. Complete End-to-End Workflow & Pipeline

The system functions through a 7-stage real-time execution loop executed every ~33 milliseconds:

- **Stage 1: Video Ingestion & Skeleton Extraction:** OpenCV reads BGR video frames. MediaPipe Tasks HandLandmarker locates the hand bounding box and extracts 21 3D landmark points (x, y, z relative to camera viewport). If no hand is present, the frame gracefully outputs a 'Searching for hand' status without crashing.
- **Stage 2: Geometric Preprocessing (Normalization):** Raw pixel coordinates vary based on distance and hand position. The system performs two mathematical operations: (1) Wrist-relative translation (shifting Landmark 0 to origin), and (2) Scale normalization (dividing by the Euclidean distance between wrist 0 and middle MCP knuckle 9). This yields 63 invariant float32 features.
- **Stage 3: Random Forest Inference:** The 63 features are fed to the pre-trained Random Forest model. It evaluates probabilities across all 31 target classes in under 2ms on CPU.
- **Stage 4: Confidence Gating & Anti-Flicker Temporal Window:** If predicted confidence is below 60% (0.60), the prediction is rejected as ambiguous. A 7-frame rolling FIFO queue maintains recent predictions; only when a class wins the majority vote (>55% agreement) over 7 frames is the gesture accepted as stable.
- **Stage 5: Sentence Composition & Debouncing:** The debounced gesture token is dispatched to the SentenceBuilder. A 1.2-second cooldown timer ensures continuous hand presence does not flood identical tokens (e.g., prevents appending 'AAAAA' in 1 second). Letters seamlessly concatenate into words; spaces and backspaces allow full conversational editing.
- **Stage 6: Multi-Modal Audio Vocalization:** The assembled sentence is vocalized aloud using dual speech synthesis: either the native OS speech synthesizer via a dedicated pyttsx3 worker thread, or directly in-browser using the HTML5 Web Speech API.
- **Stage 7: Offline Neural Hindi Translation:** The composed English sentence is processed by MarianMT running locally on CPU. It tokenizes English text and generates fluent Hindi Devanagari script (e.g. 'HELLO HELP' ➔ '■■■■■■ ■■■') with zero internet connection.

```
[ Webcam Input (OpenCV) ] ■■■ [ MediaPipe HandLandmarker ] ■■■ [ 21 3D Joints ] ■ [ 63 Invariant Features ] ■■■ [ Math Normalization: Wrist Rel & Scale ] ■ ▼ [ Random Forest Classifier ] ■■■ [ Confidence Gate > 60% ] ■■■ [ 7-Frame Majority Smoothing ] ■ [ Dual Audio TTS (pyttsx3/WebSpeech) ] ■■■ [ Sentence Builder (1.2s Cooldown) ] ■■■ [ MarianMT Hindi Translation ]
```

4. Project Directory & Important Files Reference

SignBridge strictly separates presentation (frontend) from computational ML logic (backend):

File / Directory	Architectural Purpose	Key Functions / Classes	Interconnection
app.py	Root launcher & Windows runtime wrapper	Pre-loads torch, adds C-DLL paths, calls frontend_main()	Primary entry point for Streamlit
frontend/app.py	Presentation orchestrator & router	init_app_session_state(), main(), cached resource getters	Connects navbar to pages & backend
frontend/pages/live_app.py	Live recognition & practice workspace	render_live_app_page(), camera loops, telemetry cards	Consumes predictor & tracker
frontend/pages/studio.py	Dataset collection & model trainer	render_studio_page(), sample recording, train triggers	Calls DatasetManager & train_model
frontend/styles/theme.py	CSS design system & theme tokens	get_theme_css(theme), palette injection, high contrast	Injected across all UI pages
backend/preprocessing.py	Mathematical invariance & 63D features	LandmarkPreprocessor, normalize(), extract_from_landmarks()	Used by tracker, predictor, & studio
backend/hand_tracking.py	MediaPipe Tasks HandLandmarker wrapper	HandTracker, detect(), draw_landmarks()	Supplies coordinates to preprocessor
backend/predictor.py	Inference engine with confidence gating	SignPredictor, predict(), 7-frame smoothing queue	Consumes models/isl_classifier.joblib
backend/data_collection.py	Dataset manager & CSV persistence	DatasetManager, append_sample(), get_class_counts()	Reads/writes data/isl_dataset.csv
backend/text_to_speech.py	Sentence composition & dual TTS	SentenceBuilder, speak_text(), append_token()	Drives live app transcription
backend/translator.py	Offline neural English-to-Hindi MT	EnglishToHindiTranslator, translate(), load()	Consumes models/opus_mt_en_hi
frontend/utils/command_manager.py	Custom personal gesture commands	CustomCommandManager, execute_command(), JSON storage	Triggers macros on HELP/HELLO

5. Source Code Deep Dive & Mathematical Formulations

5.1 Mathematical Normalization (backend/preprocessing.py)

Raw coordinates from computer vision vary dramatically if a signer sits 1 meter vs 2 meters from the camera, or shifts from left to right. Normalization solves this:

1. **Translation Invariance (Shift to Origin):** Let Landmark 0 be the wrist joint with coordinates P_0 = (x_0, y_0, z_0). For all 21 landmarks P_i (where i = 0 ... 20), we compute:
   P'_i = P_i - P_0
   This makes the wrist the coordinate origin (0, 0, 0), eliminating hand placement position from the equation.
2. **Scale Invariance (Normalized Hand Span):** The distance from the wrist (Joint 0) to the middle finger MCP joint (Joint 9) represents hand size. We compute the Euclidean norm:
   d_scale = || P_9 - P_0 || = sqrt((x_9 - x_0)^2 + (y_9 - y_0)^2 + (z_9 - z_0)^2)
   All translated points are normalized by this distance: **P''_i = P'_i / d_scale**. (With safety epsilon 1e-4 preventing division by zero).
3. **Flattening:** The resulting 21 3D points are flattened into a 1D float32 array: [lm_0_x, lm_0_y, lm_0_z, ..., lm_20_z] containing exactly **63 features**.

5.2 Anti-Flicker Temporal Window & Confidence Gating (backend/predictor.py)

Single-frame computer vision predictions suffer from high-frequency jitter (rapidly toggling between visually similar letters like 'M' and 'N'). SignBridge solves this through a dual-stage filter:

1. **Confidence Threshold Gating:** The Random Forest outputs probability vector P over all 31 classes. If max(P) < 0.60, the prediction is classified as invalid ('Low confidence - hold steady').
2. **Majority Voting Smoothing:** Valid predictions enter a FIFO deque of size W=7. The final reported label is:
   Label_final = mode(queue) if **count(mode(queue)) / W >= 0.55**
   This guarantees that only stable gestures held for at least 4-5 consecutive frames (~150ms) are recognized.

5.3 Debounced Sentence Composition (backend/text_to_speech.py)

A video stream processes ~30 frames/sec. If a signer displays the letter 'A' for 1 second, a naive system would produce 'AAAAAAAAAAAAAAAAAAAAAAAAAAAAAA'. SignBridge uses a **1.2-second cooldown debouncer**:

Debounce Rule: If token == last_token, require **(current_time - last_added_time) >= 1.2 seconds** before re-appending. If the user transitions to a different sign, the cooldown immediately resets.

5.4 Offline MarianMT Neural Translation (backend/translator.py)

Offline translation loads the MarianMT Seq2Seq Transformer model from `models/opus_mt_en_hi`. It executes entirely on CPU via PyTorch. English sentences are tokenized with SentencePiece, passed through 6 encoder-decoder layers, and decoded into Devanagari Hindi using beam search.

6. Machine Learning Pipeline & Dataset Metrics

Linguistic Distinction (ISL vs ASL): American Sign Language (ASL) uses a single-handed manual alphabet derived from French Sign Language. Indian Sign Language (ISL), officially codified by ISLRTC, traditionally uses a two-handed manual alphabet. For single-camera desktop compatibility, dominant hand configurations and standard ISL gestures are recorded.

Trained Classes: 31 Classes total — 26 Alphabet letters (A to Z) + 5 Core Words (HELLO, HELP, NO, THANK_YOU, YES).

Metric	Value	Significance for Project Viva
Total Dataset Samples	2,093 samples	Real recorded geometric landmarks stored in CSV
Number of Estimators	150 Decision Trees	High variance reduction without overfitting
Test Accuracy	**99.05%**	Stratified evaluation across all 31 gesture classes
Macro Precision	**99.09%**	Low false-positive rate across every individual letter
Macro Recall	**99.50%**	Extremely low false-negative rate (rarely misses a sign)
Macro F1-Score	**99.27%**	Harmonic mean confirming balance across classes
Inference Time	~1.2 ms / sample	Leaves 96% of CPU budget for rendering and video

7. Setup, Execution & Troubleshooting Guide

Exact Steps to Run the Project on Windows 11:

1. Open PowerShell and navigate to the project directory: cd d:\vs_code\innotech
2. Launch the Streamlit application: .\.venv\Scripts\streamlit.exe run app.py
3. The browser will automatically open to http://localhost:8501. Navigate to 'Live App' and allow camera access.

4. To execute the automated verification test suite: .\.venv\Scripts\pytest.exe -v (52 passed in ~37s).

Common Error / Issue	Root Cause	Solution Applied
WinError 4551 (SAC Block)	Windows Smart App Control blocks unsigned dev wheels	Pinned pandas==2.2.3 and added os.add_dll_directory in app.py
WinError 1114 (DLL init fail)	OpenMP/c10 conflict between PyTorch & MediaPipe	Pre-imported `torch` in app.py before OpenCV/MediaPipe
Webcam screen is black	Camera index 0 occupied by Zoom/Teams/Browser	Close other camera apps or switch camera index in settings

8. College Presentation Script & Pitch Guide

8.1 The 1-Minute Elevator Pitch

■ 1-Minute Project Pitch

"Good morning respected professors and evaluators. Our project is **SignBridge** — an AI-powered assistive communication platform that translates Indian Sign Language into real-time speech and Hindi text.

Over 63 million people in India have hearing loss, but less than 1% of hearing people know sign language. Existing apps rely on expensive cloud APIs that compromise privacy and fail without internet. SignBridge solves this by running **100% locally on a standard laptop webcam**. It tracks 21 hand joints via MediaPipe, normalizes them for hand size and camera distance, classifies 31 signs using a 99% accurate Random Forest model, speaks sentences aloud, and translates them into Hindi using a local neural model. Thank you!"

8.2 Detailed 3–5 Minute Presentation Outline

Slide 1: Title & Problem: Introduce SignBridge. Highlight the 63M Indian population with auditory impairment and the acute lack of human interpreters.

Slide 2: System Architecture: Walk through the 7-stage pipeline: Webcam ➔ MediaPipe ➔ Math Normalization (63 features) ➔ Random Forest ➔ Temporal Smoothing ➔ Dual TTS & MarianMT Hindi.

Slide 3: Technical Highlights: Explain Wrist-Relative Translation & Scale Invariance. Emphasize why geometric landmarks are superior to raw CNN pixels (immune to skin tone and background lighting).

Slide 4: Live Demonstration: Show fingerspelling forming a word, sentence controls (Space/Backspace), audio synthesis, and offline Hindi translation.

Slide 5: Metrics & Results: Present 99.05% accuracy across 2,093 samples and 52 passed unit tests. Conclude with zero cloud costs and complete privacy.

9. Master Viva Voce Preparation (35 Questions & Answers)

Below are the exact questions professors, external examiners, and technical evaluators ask, complete with crisp, bulletproof answers:

Q1: What is the primary objective of SignBridge?

SignBridge enables two-way communication between deaf/mute individuals using Indian Sign Language and hearing individuals by converting hand signs into real-time spoken audio and offline Hindi translation.

Q2: Why did you choose MediaPipe over training a raw CNN (like ResNet or YOLO)?

A raw CNN is sensitive to background clutter, room lighting, skin tone, and clothing. MediaPipe abstracts the hand into 21 pure geometric joint coordinates, eliminating visual bias, reducing memory footprint, and running at 30+ FPS on standard CPU without GPUs.

Q3: What are the 63 features used in classification?

MediaPipe outputs 21 landmarks, each with 3 spatial coordinates (x, y, z). 21 × 3 = 63 numerical features representing the spatial skeleton of the hand.

Q4: How do you achieve Translation Invariance?

We subtract the wrist coordinate (Landmark 0) from all other 20 landmarks: P'_i = P_i - P_0. This shifts the hand's reference origin to (0,0,0) regardless of where the hand is positioned on the camera screen.

Q5: How do you achieve Scale Invariance?

We compute the Euclidean distance between the wrist (Landmark 0) and the middle finger MCP joint (Landmark 9). All coordinates are divided by this distance, so hand size or distance from the camera does not alter the geometric representation.

Q6: Why Random Forest instead of an SVM, KNN, or Deep Neural Network?

For a tabular 63-feature dataset, Random Forest trains in seconds, handles non-linear relationships across joint angles effortlessly, does not require feature scaling, provides class probabilities, and resists overfitting through bagging across 150 trees.

Q7: How many classes can your trained model recognize?

31 classes: all 26 manual alphabet letters (A to Z) plus 5 core Indian Sign Language words (HELLO, HELP, NO, THANK_YOU, YES).

Q8: What is Confidence Gating and why is it set to 60%?

If the Random Forest's highest predicted probability is under 0.60, the system rejects the prediction as ambiguous. This prevents random guessing when the user transitions between signs or moves their hand.

Q9: What is the purpose of the 7-Frame Rolling Window?

Single-frame video predictions can flicker between similar letters due to micro-movements. The 7-frame queue takes a majority vote; a sign is only accepted if it wins >=55% of the last 7 frames, ensuring stable recognition.

Q10: What is the Cooldown Debouncer in the SentenceBuilder?

At 30 FPS, holding a hand for 1 second produces 30 detections. The 1.2s cooldown timer prevents repeating the same token repeatedly while allowing instant appending of new, distinct tokens.

Q11: How does the offline Hindi translation work without internet?

We package a local Hugging Face MarianMT model (Helsinki-NLP/opus-mt-en-hi) in the models/ directory. A Seq2Seq Transformer model runs locally on CPU with zero network calls, converting English tokens into Devanagari Hindi.

Q12: Why do you have a 'Dual TTS' engine?

pyttsx3 runs on a native background OS thread for desktop audio, while HTML5 Web Speech API synthesizes audio directly inside the web browser. If one fails or lacks system drivers, the other serves as an automatic fallback.

Q13: How did you prevent Data Leakage during model training?

We use GroupShuffleSplit based on 'session_id'. Samples from the same recording session never appear in both training and test sets, ensuring the model generalizes to unseen sessions rather than memorizing a specific recording.

Q14: What is the difference between ISL and ASL?

American Sign Language (ASL) uses a single-handed alphabet based on French Sign Language. Indian Sign Language (ISL), codified by ISLRTC, traditionally utilizes two-handed alphabets based on the British BANZSL family. SignBridge adapts single-handed dominant postures for single-camera desktop convenience.

Q15: How does the Studio page work?

The Studio page allows users to record genuine ISL samples for any letter (A-Z) or word. It records 63-feature coordinates, saves metadata to data/isl_dataset.csv, and allows retraining the Random Forest model with one click.

Q16: How do you handle cases where no hand is visible?

MediaPipe returns an empty landmark list. The system catches this, resets temporal buffers, displays a 'Searching for hand...' status badge, and executes zero inference calls, conserving CPU power.

Q17: What is Streamlit session_state?

Streamlit re-executes the entire Python script upon every user interaction (button click, slider move). session_state is a persistent dictionary that preserves variables (sentence builder, active theme, history) across reruns.

Q18: What are Custom Gesture Commands?

Users can map specific gestures (like HELP or HELLO) to macro actions (like speaking an urgent emergency phrase or triggering Hindi translation) defined in data/custom_commands.json.

Q19: How do you verify code quality in SignBridge?

We maintain an automated test suite of 52 unit and integration tests using Pytest, covering preprocessing invariance, dataset schema, budget enforcement, translation, and UI logic (100% pass rate).

Q20: What are the current limitations of the project?

1. It operates primarily on single-hand signs (dominant hand). 2. It classifies static postures rather than complex dynamic signs with trajectory motion. 3. It requires reasonable camera exposure without severe motion blur.

Q21: How would you extend SignBridge for dynamic gestures (like waving or complex phrases)?

By introducing a temporal deep learning model such as an LSTM, GRU, or 1D-CNN that ingests a sequence of landmark frames (e.g. 30 consecutive frames) to classify trajectory and hand motion over time.

Q22: Why did you create separate frontend and backend directories?

Architectural separation of concerns: backend modules can be imported into mobile apps, CLI scripts, or REST APIs without depending on Streamlit UI code.

Q23: What is the purpose of the src/ directory?

src/ contains re-export shims that point to backend modules to ensure 100% backwards compatibility with earlier scripts and tests.

Q24: How much memory (RAM) does SignBridge consume?

The Random Forest model is ~11 MB, MediaPipe task is ~8 MB, and the MarianMT model is ~300 MB. Total operational memory footprint is under 600 MB of RAM, easily running on 4GB or 8GB laptops.

Q25: What is the time complexity of the Random Forest inference?

O(M * D), where M is the number of trees (150) and D is the maximum depth of each tree (~15-20). This takes approximately 1.2 milliseconds on CPU.

Q26: What is Zip Slip and how did you protect against it?

Zip Slip is a path traversal vulnerability where malicious zip files extract files outside the target directory using '../'. We implemented canonical path validation in prepare_dataset.py.

Q27: What is the role of joblib?

Joblib serializes Python objects with large NumPy arrays efficiently using memory mapping, enabling instantaneous loading of our trained model.

Q28: How does the application support accessibility in its UI?

High-contrast color palettes (light/dark mode), large scalable typography, visual status badges, and dual audio/visual feedback ensure accessibility for individuals with various sensory impairments.

Q29: Can this model recognize signs from someone wearing gloves or jewelry?

Yes, because MediaPipe's neural network tracks skeletal joint structures and depths rather than relying on skin-tone color segmentations.

Q30: What happens if two hands enter the webcam frame?

HandTracker is initialized with max_num_hands=1, prioritizing the primary/dominant hand with highest detection score, preventing feature dimension mismatch.

Q31: What is MarianTokenizer?

A SentencePiece subword tokenizer that converts English character strings into numerical token IDs mapped to the MarianMT bilingual vocabulary dictionary.

Q32: Why did you include automated py_compile checks?

py_compile checks Python bytecode compilation across all 47 project files to guarantee zero syntax errors before deployment or test execution.

Q33: How does Practice Mode help learners?

Practice Mode prompts users with target alphabet letters, compares real-time signs against the target, and provides instant visual feedback and score tracking.

Q34: How is model accuracy evaluated?

Through stratified train-test splitting and cross-validation, computing accuracy, precision, recall, and F1-score across all 31 classes (current accuracy: 99.05%).

Q35: What is the future vision for SignBridge?

Deploying as a lightweight mobile app, integrating two-hand continuous ISL recognition via temporal Transformers, and incorporating two-way speech-to-sign 3D avatar animations.